

强化学习笔记

马尔可夫决策过程 (MDP)

MDP的基本概念

马尔可夫性质

- 定义：当前状态 S_t 完全决定下一状态 S_{t+1} ，与历史无关：

$$P(S_{t+1}|S_t) = P(S_{t+1}|S_1, \dots, S_t)$$

- 特点：
 - 当前状态是未来状态的充分统计量。
 - 历史信息通过当前状态间接传递。

马尔可夫过程

- 定义：具有马尔可夫性质的随机过程，也称为马尔可夫链。
- 表示：用元组 $(\mathcal{S}, \mathcal{P})$ 描述：
 - $\mathcal{S}$ ：状态集合 $\{s_1, s_2, \dots, s_n\}$ 。
 - $\mathcal{P}$ ：状态转移矩阵。
- 状态转移矩阵：

$$\mathcal{P} = \begin{bmatrix} P(s_1|s_1) & P(s_2|s_1) & \cdots & P(s_n|s_1) \\ P(s_1|s_2) & P(s_2|s_2) & \cdots & P(s_n|s_2) \\ \vdots & \vdots & \ddots & \vdots \\ P(s_1|s_n) & P(s_2|s_n) & \cdots & P(s_n|s_n) \end{bmatrix}$$

- 元素 $P(s_j|s_i)$ 表示从 s_i 转移到 s_j 的概率。
- 每行概率和为 1：

$$\sum_{j=1}^n P(s_j|s_i) = 1$$

- 终止状态：某些状态（如 s_6 ）以概率 1 转移到自身，不再转移。

马尔可夫奖励过程

- **定义**：在马尔可夫过程的基础上加入奖励函数 $R(s)$ 和折扣因子 γ 。
- **奖励函数**： $R(s)$ 表示在状态 s 下的即时奖励。
- **折扣因子**： γ 用于平衡即时奖励和未来奖励的重要性。

回报与价值函数

- **回报**：从时刻 t 开始，所有未来奖励的折扣和。

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

- **价值函数**：从状态 s 开始的期望回报。

$$V(s) = E[G_t | S_t = s]$$

MDP中的策略与价值函数

策略

1. 定义：

- 策略 π 是一个函数，表示在状态 s 下采取动作 a 的概率：

$$\pi(a|s) = P(A_t = a | S_t = s)$$

- **确定性策略**：每个状态下只输出一个确定性的动作（概率为 1）。
- **随机性策略**：每个状态下输出动作的概率分布，根据分布采样得到动作。
- 由于马尔可夫性质，策略仅依赖当前状态，无需考虑历史状态。

状态价值函数

1. 定义：

- 状态价值函数 $V^\pi(s)$ 表示从状态 s 出发，遵循策略 π 能获得的期望回报：

$$V^\pi(s) = \mathbb{E}[G_t | S_t = s, \pi]$$

- 不同策略在同一状态下的价值可能不同，因为策略决定了动作选择和后续状态转移。

动作价值函数

1. 定义：

- 动作价值函数 $Q^\pi(s, a)$ 表示在状态 s 下执行动作 a 并遵循策略 π 能获得的期望回报：

$$Q^\pi(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a, \pi]$$

2. 关系：

- 状态价值函数与动作价值函数的关系：

$$V^\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) Q^\pi(s, a)$$

- 动作价值函数的递推关系：

$$Q^\pi(s, a) = r(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a) V^\pi(s')$$

贝尔曼期望方程

基本定义

首先明确强化学习中的几个核心概念：

- **状态值函数** $V^\pi(s)$ ：表示从状态 s 开始，遵循策略 π 的期望回报。

$$V^\pi(s) = \mathbb{E}_\pi [G_t | S_t = s]$$

其中 G_t 是从时间 t 开始的累积折扣回报：

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

- **动作值函数** $Q^\pi(s, a)$ ：表示在状态 s 下执行动作 a 后，遵循策略 π 的期望回报。

$$Q^\pi(s, a) = \mathbb{E}_\pi [G_t | S_t = s, A_t = a]$$

- **策略** $\pi(a|s)$ ：在状态 s 下选择动作 a 的概率。
- **状态转移概率** $P(s'|s, a)$ ：在状态 s 执行动作 a 后转移到状态 s' 的概率。

- **即时奖励** $R(s, a)$: 在状态 s 执行动作 a 后的即时奖励（为简化推导，假设 $R(s, a)$ 是确定的）。
-

状态值函数 $V^\pi(s)$ 的贝尔曼期望方程

我们从 $V^\pi(s)$ 的定义出发：

$$V^\pi(s) = \mathbb{E}_\pi [G_t \mid S_t = s]$$

将 G_t 展开为即时奖励和未来回报：

$$G_t = R_{t+1} + \gamma G_{t+1}$$

因此：

$$V^\pi(s) = \mathbb{E}_\pi [R_{t+1} + \gamma G_{t+1} \mid S_t = s]$$

根据期望的线性性质：

$$V^\pi(s) = \mathbb{E}_\pi [R_{t+1} \mid S_t = s] + \gamma \mathbb{E}_\pi [G_{t+1} \mid S_t = s]$$

(1) 计算第一项： $\mathbb{E}_\pi [R_{t+1} \mid S_t = s]$

这是从状态 s 出发的即时奖励的期望。由于策略 π 决定了动作的选择，而动作又影响奖励和状态转移，因此：

$$\mathbb{E}_\pi [R_{t+1} \mid S_t = s] = \sum_a \pi(a|s) \cdot R(s, a)$$

(2) 计算第二项： $\mathbb{E}_\pi [G_{t+1} \mid S_t = s]$

这是从状态 s 出发的未来回报的期望。由于 G_{t+1} 依赖于下一状态 S_{t+1} ，而 S_{t+1} 又由动作 A_t 和状态转移概率决定：

$$\mathbb{E}_\pi [G_{t+1} \mid S_t = s] = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) \mathbb{E}_\pi [G_{t+1} \mid S_{t+1} = s']$$

注意到 $\mathbb{E}_\pi [G_{t+1} \mid S_{t+1} = s'] = V^\pi(s')$ ，因此：

$$\mathbb{E}_\pi [G_{t+1} \mid S_t = s] = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) V^\pi(s')$$

(3) 合并两项

将两部分代入 $V^\pi(s)$ 的表达式：

$$V^\pi(s) = \sum_a \pi(a|s) R(s, a) + \gamma \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) V^\pi(s')$$

合并同类项：

$$V^\pi(s) = \sum_a \pi(a|s) \left(R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^\pi(s') \right)$$

这就是**状态值函数的贝尔曼期望方程**。

动作值函数 $Q^\pi(s, a)$ 的贝尔曼期望方程

我们从 $Q^\pi(s, a)$ 的定义出发：

$$Q^\pi(s, a) = \mathbb{E}_\pi [G_t \mid S_t = s, A_t = a]$$

同样展开 G_t ：

$$G_t = R_{t+1} + \gamma G_{t+1}$$

因此：

$$Q^\pi(s, a) = \mathbb{E}_\pi [R_{t+1} + \gamma G_{t+1} \mid S_t = s, A_t = a]$$

根据期望的线性性质：

$$Q^\pi(s, a) = \mathbb{E}_\pi [R_{t+1} \mid S_t = s, A_t = a] + \gamma \mathbb{E}_\pi [G_{t+1} \mid S_t = s, A_t = a]$$

(1) 计算第一项： $\mathbb{E}_\pi [R_{t+1} \mid S_t = s, A_t = a]$

由于动作 a 已确定，直接等于即时奖励：

$$\mathbb{E}_\pi [R_{t+1} \mid S_t = s, A_t = a] = R(s, a)$$

(2) 计算第二项： $\mathbb{E}_\pi [G_{t+1} \mid S_t = s, A_t = a]$

G_{t+1} 依赖于下一状态 S_{t+1} ，而 S_{t+1} 由状态转移概率决定：

$$\mathbb{E}_\pi [G_{t+1} \mid S_t = s, A_t = a] = \sum_{s'} P(s'|s, a) \mathbb{E}_\pi [G_{t+1} \mid S_{t+1} = s']$$

$\mathbb{E}_\pi [G_{t+1} \mid S_{t+1} = s']$ 是从状态 s' 开始的期望回报，但需要遵循策略 π ，因此：

$$\mathbb{E}_\pi [G_{t+1} \mid S_{t+1} = s'] = \sum_{a'} \pi(a'|s') Q^\pi(s', a')$$

(3) 合并两项

将两部分代入 $Q^\pi(s, a)$ 的表达式：

$$Q^\pi(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) \sum_{a'} \pi(a'|s') Q^\pi(s', a')$$

这就是**动作值函数的贝尔曼期望方程**。

MDP的求解方法

动态规划 (DP)

- **策略评估**: 计算给定策略 π 下的价值函数 $V_\pi(s)$, 使用贝尔曼期望方程的迭代方法。
- **策略改进**: 在当前策略价值函数的基础上, 寻找更好的策略, 最终收敛到最优策略 π^* 。

蒙特卡洛 (MC) 方法

- **采样**: 按照策略 π 和环境的转移概率 P 反复模拟状态转移序列。
- **价值函数估计**: 对于每个访问过的状态 s , 计算所有采样序列中从 s 开始的回报 G_t 的平均值作为其估计价值。

$$V_\pi(s) \approx \frac{1}{N(s)} \sum_{i=1}^{N(s)} G_t^{(i)}$$

时序差分 (TD) 学习

- **定义**: 结合了MC方法和DP方法的思想, 只使用单个时间步的转移来进行价值函数的增量更新。
- **更新公式**:

$$V_\pi(s) \leftarrow V_\pi(s) + \alpha [R(s, a, s') + \gamma V_\pi(s') - V_\pi(s)]$$

最优策略

目标

- 强化学习的目标是找到一个策略, 使得智能体从初始状态出发能获得最多的期望回报。

策略的偏序关系

- **定义**：对于任意状态 s ，如果策略 π 的价值函数满足 $V^\pi(s) \geq V^{\pi'}(s)$ ，则记 $\pi \geq \pi'$ 。
- **最优策略**：在有限状态和动作集合的 MDP 中，至少存在一个策略不差于其他所有策略，称为最优策略 π^* 。

最优价值函数

1. 最优状态价值函数：

$$V^*(s) = \max_{\pi} V^\pi(s), \quad \forall s \in \mathcal{S}$$

表示在所有策略中，状态 s 的最大价值。

2. 最优动作价值函数：

$$Q^*(s, a) = \max_{\pi} Q^\pi(s, a), \quad \forall s \in \mathcal{S}, a \in \mathcal{A}$$

表示在所有策略中，状态动作对 (s, a) 的最大价值。

最优价值函数的关系

1. 最优状态价值与最优动作价值：

$$V^*(s) = \max_{a \in \mathcal{A}} Q^*(s, a)$$

2. 最优动作价值的递推关系：

$$Q^*(s, a) = r(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a) V^*(s')$$

贝尔曼最优方程

1. 状态价值的贝尔曼最优方程：

$$V^*(s) = \max_{a \in \mathcal{A}} \left\{ r(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a) V^*(s') \right\}$$

2. 动作价值的贝尔曼最优方程：

$$Q^*(s, a) = r(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a) \max_{a' \in \mathcal{A}} Q^*(s', a')$$

蒙特卡罗方法 (Monte Carlo, MC)

核心思想

- 目标：**通过与环境交互的完整回合 (episode) 数据，直接估计值函数（状态值 $V(s)$ 或动作值 $Q(s, a)$ ）。
- 核心机制：**
 - 基于完整轨迹：**必须等待一个回合结束后，利用该回合的所有经验进行更新。
 - 无模型学习：**无需环境动态模型（如转移概率 $P(s'|s, a)$ ）。
 - 经验平均：**通过对多个回合的回报 (Return) 取平均来估计值函数。

关键公式

- 回报 (Return)：**

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{T-t-1} R_T$$

其中 γ 是折扣因子， T 是回合终止时刻。

- 值函数更新**（以状态值函数为例）：

$$V(S_t) \leftarrow V(S_t) + \alpha [G_t - V(S_t)]$$

α 是学习率。

占用度量 (Occupancy Measure)

核心概念

- 定义：**占用度量描述在某个策略 π 下，智能体在环境中访问状态或状态-动作对的长期分布。
- 数学形式：**
 - 状态占用度量 $\rho_\pi(s)$ ：**表示在策略 π 下，状态 s 的长期访问概率。
 - 状态-动作占用度量 $\rho_\pi(s, a)$ ：**表示在策略 π 下，状态-动作对 (s, a) 的长期访问概率。

数学表达式

- 折扣占用度量 (考虑折扣因子 γ) :

$$\rho_{\pi}(s) = \sum_{t=0}^{\infty} \gamma^t P_{\pi}(S_t = s)$$

其中 $P_{\pi}(S_t = s)$ 是策略 π 下时刻 t 处于状态 s 的概率。

- 未折扣占用度量 ($\gamma = 1$, 但需环境为遍历性) 。

占用度量的性质

- 归一性: 若 $\gamma < 1$, 则 $\sum_s \rho_{\pi}(s) = \frac{1}{1-\gamma}$ 。
- 策略依赖性: 不同策略 π 会产生不同的占用度量。
- 动态规划关联: 占用度量满足贝尔曼方程:

$$\rho_{\pi}(s, a) = d_0(s)\pi(a|s) + \gamma \sum_{s', a'} \rho_{\pi}(s', a') P(s|s', a') \pi(a|s)$$

其中 $d_0(s)$ 是初始状态分布。

动态规划算法

动态规划的核心思想

动态规划 (Dynamic Programming, DP) 是一种通过**分解问题**、**保存子问题解**并**递推求解全局解**的高效算法设计方法，其核心思想可概括为以下三点：

1. 子问题分解

将复杂问题分解为多个相互关联的**子问题**，通过递归或迭代逐步求解子问题，最终合并得到原问题的解。

2. 重叠子问题与记忆化存储

子问题之间可能存在**重叠计算**，动态规划通过**存储已解决的子问题答案**（记忆化），避免重复计算，显著提升效率。

3. 最优子结构

全局最优解可通过局部最优解递推得到，即**当前决策仅依赖子问题的最优解**，与后续决策无关。

核心特点与局限

1. 优势

- 高效性**：利用子问题解复用，避免无模型方法（如蒙特卡洛）的大量采样。
- 精确性**：在已知环境模型（状态转移 P 和奖励 R ）时理论收敛性严格。

2. 局限

- 依赖环境模型**：需完全已知MDP（白盒环境），难以适应实际黑盒场景。
 - 维数灾难**：仅适用于离散有限状态空间，连续/高维问题需结合函数近似。
 - 同步更新限制**：需全局遍历状态，无法在线学习。
-

策略迭代 (Policy Iteration)

策略迭代的核心思想

策略迭代的核心分为两步：

- 策略评估**：计算当前策略 π 的状态值函数 $V^\pi(s)$ 。
- 策略改进**：基于 $V^\pi(s)$ 更新策略 π ，使其更优（即选择使 $V^\pi(s)$ 最大的动作）。

通过不断迭代这两个步骤，策略会逐渐收敛到最优策略 π^* 。

策略迭代的算法流程

1. 初始化：

- 随机初始化策略 π （如均匀随机策略）。
- 初始化状态值函数 $V(s)$ （如全零）。

2. 策略评估 (Policy Evaluation)：

- 计算当前策略 π 的状态值函数 $V^\pi(s)$ ：

$$V^\pi(s) = \sum_a \pi(a|s) \left(R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^\pi(s') \right)$$

- 通过迭代法求解（如贝尔曼期望方程的迭代计算）：

$$V_{k+1}(s) = \sum_a \pi(a|s) \left(R(s, a) + \gamma \sum_{s'} P(s'|s, a) V_k(s') \right)$$

直到 $V(s)$ 收敛（即 $|V_{k+1}(s) - V_k(s)| < \epsilon$ ）。

3. 策略改进 (Policy Improvement)：

- 基于当前 $V^\pi(s)$ ，更新策略 π ：

$$\pi'(s) = \arg \max_a \left(R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^\pi(s') \right)$$

- 即对每个状态 s ，选择使 $Q^\pi(s, a)$ 最大的动作 a ：

$$\pi'(s) = \arg \max_a Q^\pi(s, a)$$

- 如果 $\pi' = \pi$ ，则算法终止；否则，返回步骤 2。

策略迭代的伪代码

```
1 def policy_iteration(MDP, gamma=0.9, epsilon=1e-6):
2     # 初始化
3     V = {s: 0 for s in MDP.states}
4     pi = {s: random_action(MDP.actions) for s in MDP.states}
5
6     while True:
7         # 策略评估
8         while True:
9             delta = 0
10            for s in MDP.states:
11                v_old = V[s]
12                V[s] = sum(pi[a|s] * (MDP.R(s, a) + gamma *
13sum(MDP.P(s'|s, a) * V[s'])
14                                for s' in MDP.states)) for a in
15MDP.actions)
16                delta = max(delta, abs(v_old - V[s]))
17            if delta < epsilon:
18                break
19
20        # 策略改进
21        policy_stable = True
22        for s in MDP.states:
23            old_action = pi[s]
24            pi[s] = argmax_a(MDP.R(s, a) + gamma *
25sum(MDP.P(s'|s, a) * V[s'])
26                                for s' in MDP.states))
27            if old_action != pi[s]:
28                policy_stable = False
29
30        if policy_stable:
31            break
32
33    return V, pi
```

价值迭代 (Value Iteration)

核心思想

价值迭代基于**贝尔曼最优方程 (Bellman Optimality Equation)**：

$$V^*(s) = \max_a \left(R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^*(s') \right)$$

- 目标**：通过迭代更新 $V(s)$ ，使其收敛到最优值函数 V^* 。
- 策略提取**：收敛后，通过贪心选择动作得到最优策略：

$$\pi^*(s) = \arg \max_a \left(R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^*(s') \right)$$

算法流程

1. 初始化：

- 对所有状态 $s \in S$ ，设 $V_0(s) = 0$ (或其他初始值)。

2. 迭代更新值函数：

- 对每次迭代 k ，更新所有状态 s ：

$$V_{k+1}(s) = \max_a \left(R(s, a) + \gamma \sum_{s'} P(s'|s, a) V_k(s') \right)$$

- 计算值函数的变化量：

$$\Delta = \max_s |V_{k+1}(s) - V_k(s)|$$

- 若 $\Delta < \epsilon$ ，停止迭代；否则继续。

3. 提取最优策略：

- 对每个状态 s ，选择使值最大化的动作：

$$\pi^*(s) = \arg \max_a \left(R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^*(s') \right)$$

伪代码实现

```
1 def value_iteration(MDP, gamma=0.9, epsilon=1e-6):
2     # 初始化值函数
3     V = {s: 0 for s in MDP.states}
4
5     while True:
6         delta = 0
7         for s in MDP.states:
8             v_old = V[s]
9             # 计算所有动作的Q值，并选择最大值
10            V[s] = max(MDP.R(s, a) + gamma * sum(MDP.P(s'|s, a)
11            * V[s'])
12                       for a in MDP.actions for s' in MDP.states)
13            delta = max(delta, abs(v_old - V[s]))
14        if delta < epsilon:
15            break
16
17    # 提取最优策略
18    pi = {}
19    for s in MDP.states:
20        pi[s] = argmax_a(MDP.R(s, a) + gamma * sum(MDP.P(s'|s,
21        a) * V[s'])
22                       for s' in MDP.states))
23
24    return V, pi
```

动态规划思想的核心体现

最优子结构

- **策略迭代**：策略改进阶段选择局部最优动作，逐步逼近全局最优策略。
- **价值迭代**：每一步价值更新均基于后续状态的最优值，直接追求全局最优。

无后效性

- 状态价值的更新仅依赖当前状态和后续状态的最优值，与历史路径无关。

算法对比与收敛性

特性	策略迭代	价值迭代
核心理念	分阶段优化：策略评估→策略改进	直接优化价值函数，隐式优化策略
数学形式	贝尔曼期望方程 + 贪婪策略改进	贝尔曼最优方程 + 最大值操作
计算复杂度	策略评估需迭代解线性方程组 ($O(n^2)$ /次)	每次迭代为矩阵乘法和逐元素最大值 ($O(n^2 \cdot A)$ /次)
收敛速度	策略变化可能剧烈，收敛速度依赖策略改进幅度	受折扣因子 γ 影响，速率稳定
适用场景	策略需显式维护或频繁变化时	状态空间较大或需快速收敛时

动态规划的局限性

- 维数灾难**：状态空间 $|S|$ 增大时，矩阵存储与计算成本指数级增长。
 - 依赖完整环境模型**：需已知转移概率 $P(s'|s, a)$ 和奖励函数 $R(s, a)$ ，实际中常需采样估计。
 - 同步更新限制**：传统动态规划需遍历所有状态，难以适应在线学习或部分可观测场景。
-

时序差分

时序差分 (TD) 算法核心思想

基本定义

- 无模型 (Model-Free)**：无需环境动态模型，直接通过交互数据学习。
- 自举 (Bootstrap)**：结合当前估计值更新目标值，融合动态规划和蒙特卡洛思想。
- 单步更新**：每一步交互后立即更新值函数，无需等待完整轨迹。

核心优势对比

方法	更新延迟	偏差-方差权衡	适用场景
蒙特卡洛	高 (需完整轨迹)	无偏, 高方差	回合制任务 (如棋类)
动态规划	低 (全状态批量更新)	无偏, 无方差	已知模型的规划任务
时序差分	低 (单步更新)	有偏, 低方差	连续交互任务 (如机器人控制)

核心思想

TD 算法的核心是**利用部分真实观测和当前估计**来更新值函数，其更新公式为：

$$V(s_t) \leftarrow V(s_t) + \alpha [r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$$

其中：

- $r_{t+1} + \gamma V(s_{t+1})$ ：称为 **TD 目标**，结合了即时奖励和后续状态的估计值。
- $r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$ ：称为 **TD 误差 (δ)**，衡量当前估计与目标的差距。
- α ：学习率，控制更新步长。

数学基础与推导

贝尔曼方程

- 状态值函数:

$$V^{\pi}(s) = \mathbb{E}_{\pi} [R_{t+1} + \gamma V^{\pi}(S_{t+1}) \mid S_t = s]$$

- 动作值函数:

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi} [R_{t+1} + \gamma Q^{\pi}(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a]$$

TD误差与更新规则

- TD误差:

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t) \quad \text{或} \quad \delta_t = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)$$

- 值函数更新:

$$V(S_t) \leftarrow V(S_t) + \alpha \delta_t, \quad Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \delta_t$$

- α 为学习率, 控制更新步长。

收敛性分析 (TD(0))

- 假设条件:

- 有限MDP (状态和动作空间有限)。
- 策略固定 (策略评估阶段)。
- 学习率满足Robbins-Monro条件:

$$\sum_{t=1}^{\infty} \alpha_t = \infty, \quad \sum_{t=1}^{\infty} \alpha_t^2 < \infty$$

- 证明思路:

- 随机近似理论:** 将TD更新视为随机逼近过程, 最小化均方投影误差。
 - 压缩映射:** 贝尔曼算子 T^{π} 是压缩映射, 保证收敛到唯一不动点 V^{π} 。
-

TD 算法的分类

(1) TD(0) (单步 TD)

- 更新公式:

$$V(s_t) \leftarrow V(s_t) + \alpha [r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$$

- 特点: 仅使用下一步的奖励和值函数估计。

(2) 多步 TD (TD(n))

- 更新公式:

$$V(s_t) \leftarrow V(s_t) + \alpha \left[\sum_{k=1}^n \gamma^{k-1} r_{t+k} + \gamma^n V(s_{t+n}) - V(s_t) \right]$$

- 特点: 平衡 TD(0) 的偏差和蒙特卡洛的方差。

(3) TD(λ) (资格迹)

- 思想: 加权结合所有步长的 TD 误差。

- 前向视角:

$$V(s_t) \leftarrow V(s_t) + \alpha [G_t^\lambda - V(s_t)]$$

其中 $G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$ 。

- 后向视角 (更实用):

- 使用**资格迹 (eligibility trace)** $e_t(s)$ 跟踪状态访问频率。
- 更新规则:

$$V(s) \leftarrow V(s) + \alpha \delta_t e_t(s)$$

$$e_t(s) = \begin{cases} \gamma \lambda e_{t-1}(s) + 1 & \text{if } s = s_t \\ \gamma \lambda e_{t-1}(s) & \text{otherwise} \end{cases}$$

SARSA

核心思想

SARSA 的核心是通过 **贝尔曼期望方程** 更新动作值函数 $Q(s, a)$, 其更新公式为:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

其中:

- α : 学习率 (步长), 控制更新幅度。
 - γ : 折扣因子, 权衡即时奖励与未来奖励。
 - $r_{t+1} + \gamma Q(s_{t+1}, a_{t+1})$: TD 目标, 基于当前策略选择的下一个动作 a_{t+1} 。
 - $Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)$: TD 误差 (δ), 反映当前估计与目标的差距。
-

算法流程

1. 初始化:

- 对所有 $s \in S$ 和 $a \in A$, 随机初始化 $Q(s, a)$ 。
- 设置初始状态 s 。

2. 交互与更新:

- 在状态 s , 根据当前策略 (如 ϵ -greedy) 选择动作 a 。
- 执行动作 a , 观察到奖励 r 和下一状态 s' 。
- 在状态 s' , 再次根据当前策略选择动作 a' 。
- 更新 $Q(s, a)$:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)]$$

- 更新状态和动作: $s \leftarrow s', a \leftarrow a'$ 。
- 重复直到回合终止。

3. 策略提取:

- 最终策略为贪心策略:

$$\pi^*(s) = \arg \max_a Q(s, a)$$

伪代码实现

```
1 def sarsa(env, episodes=1000, alpha=0.1, gamma=0.99,
2   epsilon=0.1):
3     Q = defaultdict(lambda: np.zeros(env.action_space.n)) # 初
4     # 始化 Q 表
5     for _ in range(episodes):
```

```

5         s = env.reset()
6         a = epsilon_greedy(Q, s, epsilon) #  $\epsilon$ -greedy 选择动作
7
8         done = False
9         while not done:
10             s_next, r, done, _ = env.step(a) # 执行动作 a, 得到
              (s', r)
11             a_next = epsilon_greedy(Q, s_next, epsilon) # 在
              s' 选择 a'
12
13             # SARSA 更新
14             td_target = r + gamma * Q[s_next][a_next]
15             td_error = td_target - Q[s][a]
16             Q[s][a] += alpha * td_error
17
18             s, a = s_next, a_next # 更新状态和动作
19
20     return Q

```

关键特性

(1) On-Policy (同策略)

- SARSA **遵循当前策略** (如 ϵ -greedy) 选择动作 a_{t+1} , 因此学习的是当前策略的 Q 函数。
- 与 Off-Policy 的 Q-learning 不同, SARSA 更保守, 能避免危险动作 (如悬崖行走问题)。

(2) 探索与利用的平衡

- 通过 ϵ -greedy 策略:
 - 以概率 $1 - \epsilon$ 选择最优动作 (利用)。
 - 以概率 ϵ 随机探索其他动作。

(3) 收敛性

- 在 **有限状态和动作空间** 中, 若满足:
 - 所有状态-动作对被无限次访问 (通过 ϵ -greedy 保证)。
 - 学习率 α 满足 Robbins-Monro 条件 (如 $\sum \alpha = \infty$, $\sum \alpha^2 < \infty$)。
- 则 SARSA 收敛到最优 Q 函数。

变种与扩展

(1) Expected SARSA

- 改进 SARSA 的随机性，使用 **期望值** 更新：

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \sum_{a'} \pi(a'|s') Q(s', a') - Q(s, a) \right]$$

- 减少方差，比标准 SARSA 更稳定。

(2) SARSA(λ) (资格迹)

- 引入资格迹 $e(s, a)$ 加速学习：

$$Q(s, a) \leftarrow Q(s, a) + \alpha \delta e(s, a)$$

$$e(s, a) = \begin{cases} \gamma \lambda e(s, a) + 1 & \text{if } s = s_t, a = a_t \\ \gamma \lambda e(s, a) & \text{otherwise} \end{cases}$$

- 适用于稀疏奖励任务。
-

Q-learning

核心思想

Q-learning 的核心是 **贝尔曼最优方程**，其更新公式为：

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]$$

其中：

- α ：学习率（步长），控制更新幅度（通常取 0.1）。
 - γ ：折扣因子，权衡即时奖励与未来奖励（通常取 0.9）。
 - $r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a')$ ：TD 目标，基于最优动作的估计。
 - $\max_{a'} Q(s_{t+1}, a')$ ：体现“Off-Policy”特性，直接选择最优动作，而非当前策略的动作。
-

算法流程

1. 初始化:

- 对所有 $s \in S$ 和 $a \in A$, 随机初始化 $Q(s, a)$ (如全零)。
- 设置初始状态 s 。

2. 交互与更新:

- 在状态 s , 根据探索策略 (如 ϵ -greedy) 选择动作 a 。
- 执行动作 a , 观察到奖励 r 和下一状态 s' 。
- 不依赖当前策略**, 直接选择 s' 的最优动作:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

- 更新状态: $s \leftarrow s'$ 。
- 重复直到回合终止。

3. 策略提取:

- 最终策略为贪心策略:

$$\pi^*(s) = \arg \max_a Q(s, a)$$

伪代码实现

```
1 def q_learning(env, episodes=1000, alpha=0.1, gamma=0.99,
2   epsilon=0.1):
3     Q = defaultdict(lambda: np.zeros(env.action_space.n)) # 初
4     # 始化 Q 表
5
6     for _ in range(episodes):
7         s = env.reset()
8         done = False
9
10        while not done:
11            #  $\epsilon$ -greedy 选择动作
12            if np.random.rand() < epsilon:
13                a = env.action_space.sample() # 探索
14            else:
15                a = np.argmax(Q[s]) # 利用
```

```
15         # 执行动作，观察环境
16         s_next, r, done, _ = env.step(a)
17
18         # Q-learning 更新
19         td_target = r + gamma * np.max(Q[s_next])
20         td_error = td_target - Q[s][a]
21         Q[s][a] += alpha * td_error
22
23         s = s_next # 更新状态
24
25     return Q
```

关键特性

(1) Off-Policy (异策略)

- Q-learning **不依赖当前策略**选择 $\max_a Q(s', a)$ ，而是直接学习最优策略的 Q 函数。
- 允许使用任意探索策略（如 ϵ -greedy）收集数据，但优化目标独立于探索策略。

(2) 探索与利用的平衡

- 通过 ϵ -greedy 策略：
 - 以概率 $1 - \epsilon$ 选择当前最优动作（利用）。
 - 以概率 ϵ 随机探索其他动作（探索）。

(3) 收敛性

- 在 **有限状态和动作空间** 中，若满足：
 - 所有状态-动作对被无限次访问（通过 ϵ -greedy 保证）。
 - 学习率 α 满足 Robbins-Monro 条件（如 $\sum \alpha = \infty$, $\sum \alpha^2 < \infty$ ）。
 - 则 Q-learning 收敛到全局最优 Q^* 。
-

变种与扩展

(1) Double Q-learning

- 解决 Q-learning 的**过高估计问题**（因 max 操作引入偏差）。
- 使用两个独立的 Q 函数交替更新：

$$Q_1(s, a) \leftarrow Q_1(s, a) + \alpha \left[r + \gamma Q_2(s', \arg \max_{a'} Q_1(s', a')) - Q_1(s, a) \right]$$

(2) Deep Q-Network (DQN)

- 用神经网络近似 Q 函数，解决高维状态空间问题。
 - 关键技术：
 - 经验回放 (Experience Replay)**：打破数据相关性。
 - 目标网络 (Target Network)**：稳定训练。
-

Dyna-Q

核心思想

Dyna-Q 的核心是通过**真实交互数据**和**模拟经验**并行更新 Q 值：

- 真实交互**：像 Q-learning 一样与环境交互，收集真实数据 (s, a, r, s') 。
 - 模型学习**：用收集的数据构建环境模型 $\hat{P}(s'|s, a)$ 和 $\hat{R}(s, a)$ 。
 - 虚拟规划**：从模型中采样虚拟经验 $(s, a, \hat{r}, \hat{s}')$ ，反复更新 Q 值，**无需额外环境交互**。
-

算法流程

1. 初始化：

- Q 表 $Q(s, a)$ 和模型 $\hat{P}, \hat{R}$ （如空表）。
- 经验缓存 D （存储真实数据）。

2. 主循环（每回合）：

◦ 真实交互：

1. 用 ϵ -greedy 选择动作 a ，执行后得到 (s, a, r, s') 。

2. Q-learning 更新：

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

3. 模型学习:

- 记录转移 $\hat{P}(s'|s, a)$ 和奖励 $\hat{R}(s, a) = r$ 。
- 存储经验 (s, a) 到 D。

○ 虚拟规划 (循环 n 次) :

1. 从 D 随机采样一个状态-动作对 (s, a) 。
2. 从模型 $\hat{P}(s'|s, a)$ 和 $\hat{R}(s, a)$ 生成虚拟经验 $(s, a, \hat{r}, \hat{s}')$ 。

3. 虚拟 Q-learning 更新:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[\hat{r} + \gamma \max_{a'} Q(\hat{s}', a') - Q(s, a) \right]$$

伪代码实现

```

1  def dyna_q(env, episodes=1000, alpha=0.1, gamma=0.99,
    epsilon=0.1, n_planning=5):
2      Q = defaultdict(lambda: np.zeros(env.action_space.n))
3      model_P = defaultdict(dict) # 转移模型: model_P[s][a] = s'
4      model_R = defaultdict(float) # 奖励模型: model_R[(s,a)] = r
5      D = set()                    # 经验缓存
6
7      for _ in range(episodes):
8          s = env.reset()
9          done = False
10
11         while not done:
12             # 1. 真实交互 (ε-greedy)
13             if np.random.rand() < epsilon:
14                 a = env.action_space.sample()
15             else:
16                 a = np.argmax(Q[s])
17
18             s_next, r, done, _ = env.step(a)
19
20             # 2. Q-learning 更新
21             td_target = r + gamma * np.max(Q[s_next])
22             Q[s][a] += alpha * (td_target - Q[s][a])
23
24             # 3. 模型学习

```

```

25         model_P[s][a] = s_next
26         model_R[(s,a)] = r
27         D.add((s, a))
28
29         # 4. 虚拟规划 (n 次)
30         for _ in range(n_planning):
31             s_sim, a_sim = random.choice(list(D))
32             s_next_sim = model_P[s_sim][a_sim]
33             r_sim = model_R[(s_sim, a_sim)]
34
35             # 虚拟更新
36             td_target_sim = r_sim + gamma *
np.max(Q[s_next_sim])
37             Q[s_sim][a_sim] += alpha * (td_target_sim -
Q[s_sim][a_sim])
38
39             s = s_next
40
41     return Q

```

关键特性

(1) 模型学习 vs. 无模型学习

特性	Dyna-Q	Q-learning
是否需要模型	是（学习近似模型）	否
数据效率	更高（复用经验）	较低
计算开销	更高（需维护模型和规划）	较低

(2) 规划步数 n 的影响

- $n = 0$: 退化为标准 Q-learning。
- $n > 0$: 通过虚拟经验加速收敛，但增加计算成本。
- **权衡**: n 越大，学习越快，但可能过拟合模型误差。

(3) 模型准确性

- Dyna-Q 依赖模型 $\hat{P}$ 和 $\hat{R}$ 的准确性。
- 若环境随机性强或模型误差大，虚拟规划可能引入偏差。

DQN

核心思想

DQN 的核心是通过神经网络近似 Q 函数：

$$Q(s, a; \theta) \approx Q^*(s, a)$$

其中 θ 是网络参数。关键创新包括：

- 经验回放 (Experience Replay)**：打破数据相关性，提高样本效率。
- 目标网络 (Target Network)**：稳定训练，解决“移动目标”问题。

算法流程

1. 初始化：

- 随机初始化主网络参数 θ 和目标网络参数 $\theta^- = \theta$ 。
- 清空经验回放缓冲区 D 。

2. 交互与训练：

数据收集：

- 用 ϵ -greedy 策略选择动作 a_t (以 ϵ 概率随机，否则 $a_t = \arg \max_a Q(s_t, a; \theta)$)。
- 执行动作，观察到 $(s_t, a_t, r_{t+1}, s_{t+1}, \text{done})$ ，存入 D 。

训练网络：

- 从 D 随机采样一个 batch $(s_i, a_i, r_i, s_{i+1}, \text{done}_i)$ 。
- 计算目标 Q 值：

$$y_i = \begin{cases} r_i & \text{if done}_i \\ r_i + \gamma \max_{a'} Q(s_{i+1}, a'; \theta^-) & \text{otherwise} \end{cases}$$

- 更新主网络参数 θ 以最小化损失：

$$L(\theta) = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i; \theta))^2$$

- 每隔 C 步同步目标网络： $\theta^- \leftarrow \theta$ 。

关键技术与数学推导

经验回放 (Experience Replay)

- **作用：**打破数据的时间相关性，减少方差。
- **实现：**
 - 存储转移 (s_t, a_t, r_t, s_{t+1}) 到缓冲区 D 。
 - 训练时随机采样 batch，近似独立同分布数据。

目标网络 (Target Network)

- **问题：**若用同一网络计算目标和预测值，会导致“移动目标”不稳定。
- **解决方案：**
 - 使用独立的目标网络 $Q(s, a; \theta^-)$ 计算 y_i 。
 - 定期同步 $\theta^- \leftarrow \theta$ (如每 1000 步)。

损失函数

DQN 采用均方误差 (MSE) 损失：

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim D} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right]$$

梯度更新：

$$\nabla_{\theta} L(\theta) = \mathbb{E} [-2 (y - Q(s, a; \theta)) \nabla_{\theta} Q(s, a; \theta)]$$

伪代码实现

```
1 class DQN:
2     def __init__(self, state_dim, action_dim):
3         self.q_net = QNetwork(state_dim, action_dim) # 主网络
4         self.target_net = QNetwork(state_dim, action_dim) # 目标网络
5         self.buffer = ReplayBuffer(capacity=10000)
6         self.optimizer = Adam(self.q_net.parameters())
7
8     def train(self, batch_size=64, gamma=0.99):
9         states, actions, rewards, next_states, dones =
            self.buffer.sample(batch_size)
10
```

```
11         # 计算目标 Q 值
12         with torch.no_grad():
13             next_q = self.target_net(next_states).max(1)[0]
14             targets = rewards + gamma * next_q * (1 - dones)
15
16         # 计算当前 Q 值
17         current_q = self.q_net(states).gather(1, actions)
18
19         # 更新主网络
20         loss = F.mse_loss(current_q, targets)
21         self.optimizer.zero_grad()
22         loss.backward()
23         self.optimizer.step()
24
25         # 同步目标网络
26         if self.steps % C == 0:
27
28             self.target_net.load_state_dict(self.q_net.state_dict())
```

Double DQN

核心思想与动机

(1) DQN的高估问题

- **根本原因**：传统DQN使用同一个网络（目标网络）同时选择动作和计算Q值：

$$y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$$

- 由于 **max** 操作会偏向高估的Q值（因噪声或近似误差），导致学习不稳定。
- **数学解释**：设真实Q值为 Q^* ，估计误差为 ϵ ：

$$\mathbb{E}[\max_a (Q^* + \epsilon)] \geq \max_a Q^*$$

(2) Double DQN的解决方案

- **解耦动作选择和Q值评估**：
 - **动作选择**：用主网络 θ 选择最优动作。

- **Q值评估**: 用目标网络 θ^- 计算该动作的Q值。

$$y = r + \gamma Q(s', \arg \max_{a'} Q(s', a'; \theta); \theta^-)$$

- **效果**: 减少因 **max** 操作导致的高估, 提升策略的鲁棒性。
-

算法流程

1. 初始化:

- 主网络参数 θ 随机初始化, 目标网络 $\theta^- \leftarrow \theta$ 。
- 清空经验回放缓冲区 D 。

2. 交互与训练:

- **数据收集**:

1. 用 ϵ -greedy 策略选择动作 a_t 。
2. 执行动作, 观察到 $(s_t, a_t, r_t, s_{t+1}, \text{done})$, 存入 D 。

- **训练网络**:

1. 从 D 采样一个batch $(s_i, a_i, r_i, s_{i+1}, \text{done}_i)$ 。
2. 计算Double DQN目标:

$$y_i = \begin{cases} r_i & \text{if done}_i \\ r_i + \gamma Q(s_{i+1}, \arg \max_{a'} Q(s_{i+1}, a'; \theta); \theta^-) & \text{otherwise} \end{cases}$$

3. 更新主网络参数 θ 以最小化损失:

$$L(\theta) = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i; \theta))^2$$

4. 每隔 C 步同步目标网络: $\theta^- \leftarrow \theta$ 。
-

伪代码实现

```
1 class DoubleDQN:
2     def __init__(self, state_dim, action_dim):
3         self.q_net = QNetwork(state_dim, action_dim)      # 主
        网络
4         self.target_net = QNetwork(state_dim, action_dim) # 目
        标网络
5         self.optimizer = Adam(self.q_net.parameters())
6         self.buffer = ReplayBuffer(capacity=10000)
```



```

7
8     def compute_target(self, batch):
9         states, actions, rewards, next_states, dones = batch
10
11         # 主网络选择动作
12         next_actions = self.q_net(next_states).argmax(1,
keepdim=True)
13
14         # 目标网络评估Q值
15         with torch.no_grad():
16             next_q = self.target_net(next_states).gather(1,
next_actions)
17             targets = rewards + (1 - dones) * gamma * next_q
18
19         return targets
20
21     def train(self, batch_size=64):
22         batch = self.buffer.sample(batch_size)
23         targets = self.compute_target(batch)
24
25         # 计算当前Q值
26         states, actions = batch[0], batch[1]
27         current_q = self.q_net(states).gather(1, actions)
28
29         # 更新主网络
30         loss = F.mse_loss(current_q, targets)
31         self.optimizer.zero_grad()
32         loss.backward()
33         self.optimizer.step()

```

Dueling DQN

核心思想与动机

(1) 传统DQN的局限性

- **问题：**传统DQN直接输出Q值 $Q(s, a)$ ，但：
 - 某些状态下所有动作的差异很小（如赛车游戏直行时左右转无区别）。

- 某些状态下少量关键动作影响巨大（如避障时的急转弯）。
- 后果：网络难以区分状态本身的价值和动作的相对重要性。

(2) Dueling架构的创新

- 分离表示：
 - 状态价值函数 $V(s)$ ：衡量当前状态的长期收益。
 - 优势函数 $A(s, a)$ ：衡量动作 a 相对于平均水平的优势。
- 组合公式：

$$Q(s, a) = V(s) + A(s, a) - \frac{1}{|A|} \sum_{a'} A(s, a')$$

- 减去均值：保证优势函数的可辨识度（避免网络将 $V(s)$ 和 $A(s, a)$ 任意偏移）。

算法流程

1. 初始化：

- 初始化Dueling网络参数 θ 和目标网络参数 θ^- 。
- 创建经验回放缓冲区 D 。

2. 交互与训练：

- 数据收集：使用 ϵ -greedy策略与环境交互，存储数据到 D 。
- 网络更新：

1. 从 D 采样batch $(s_i, a_i, r_i, s_{i+1}, done_i)$ 。

2. 计算目标Q值（结合Double DQN）：

$$y_i = r_i + \gamma Q(s_{i+1}, \arg \max_{a'} Q(s_{i+1}, a'; \theta^-); \theta^-)$$

3. 前向传播计算当前 $Q(s_i, a_i)$ 。

4. 最小化损失 $L(\theta) = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i))^2$ 。

5. 定期同步目标网络 $\theta^- \leftarrow \theta$ 。

关键技术与数学推导

(1) 优势函数的可辨识度

- **问题：**若直接使用 $Q(s, a) = V(s) + A(s, a)$ ，网络无法唯一确定 V 和 A （如 $V + 100$ 和 $A - 100$ 等价）。
- **解决方案：**强制优势函数的均值为零：

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|A|} \sum_{a'} A(s, a') \right)$$

◦ 此时 $V(s) = \frac{1}{|A|} \sum_a Q(s, a)$ ，实现唯一解。

(2) 梯度更新分析

- **状态价值支路梯度：**

$$\nabla_{\theta} V(s) = \frac{1}{|A|} \sum_a \nabla_{\theta} Q(s, a)$$

- **优势支路梯度：**

$$\nabla_{\theta} A(s, a) = \nabla_{\theta} Q(s, a) - \frac{1}{|A|} \sum_{a'} \nabla_{\theta} Q(s, a')$$

伪代码实现

```

1  class DuelingDQN(nn.Module):
2      def __init__(self, state_dim, action_dim):
3          super().__init__()
4          self.feature_layer = nn.Sequential(
5              nn.Conv2d(4, 32, kernel_size=8, stride=4),
6              nn.ReLU(),
7              nn.Conv2d(32, 64, kernel_size=4, stride=2),
8              nn.ReLU()
9          )
10         # 状态价值支路
11         self.V = nn.Sequential(
12             nn.Linear(64 * 7 * 7, 512),
13             nn.ReLU(),
14             nn.Linear(512, 1)
15         )
16         # 优势支路
17         self.A = nn.Sequential(
18             nn.Linear(64 * 7 * 7, 512),
19             nn.ReLU(),
20             nn.Linear(512, action_dim)

```

```
21         )
22
23     def forward(self, x):
24         x = self.feature_layer(x)
25         x = x.view(x.size(0), -1)
26         V = self.V(x)
27         A = self.A(x)
28         Q = V + (A - A.mean(dim=1, keepdim=True)) # 减去均值
29         return Q
```